

3D MRI-to-Synthetic CT Translation

CS 671 — Final Evaluation

Group 7

Anish Kumar, Divyansh Jain, Chinmay Patel, Yug Goyal,
Siddhant Shah, Anshul Mendiratta, Aryan Garg, Sachit Bansal

Mentor: Munish Daroch

Repo: github.com/anishk85/MRI-TO-SCT-mamba-diffusion-gans-

Project ID: P32

Outline — 9 Approaches Evaluated

Diffusion-Based Approaches

- 1 DDPM + Swin Transformer
- 2 Pretrained MRI Encoder + Hybrid Diffusion
- 3 Diffusion U-Mamba with SSM Blocks

Deterministic Mamba Architectures

- 4 SegMamba — CNN + Mamba U-Net
- 5 U-Mamba — Conv-Mamba Hybrid Blocks
- 6 TriAxial Mamba — 3-Axis Scans
- 7 TriMamba-UNet V2 — CBAM-Gated Skips

GAN Baselines

- 8 Pix2Pix GAN (Supervised)
- 9 UNIT GAN (Unsupervised)

Presentation Flow

- 1 Problem & Background
- 2 Baseline DDPM
- 3 Pretrained MRI Encoder
- 4 The Mamba Solution
- 5 Deterministic Mamba Models
- 6 Diffusion U-Mamba
- 7 GAN Comparisons
- 8 Comprehensive Results



Problem & Background

Clinical Motivation — Why Synthetic CT?

- **Radiation Therapy (RT)** for brain tumours requires a CT scan to compute dose maps.
 - CT measures *Hounsfield Units (HU)* — a calibrated scale of tissue electron density.
 - Bone $\approx 700\text{--}1000$ HU, brain tissue $\approx 20\text{--}50$ HU, air ≈ -1000 HU.
- **The Problem:** CT exposes the patient to ionising radiation. Many patients already have an MRI for tumour delineation — a second CT scan means more cost, more radiation, and imperfect cross-modality registration.
- **Synthetic CT (sCT):** learn a mapping $f : \text{MRI} \rightarrow \text{sCT}$ so the CT can be *synthesised* directly from MRI.
- **Output range:** 3D HU volume $\in [-1024, +1650]$, behaves like a real CT scan.



Dataset — SynthRAD2023 Brain

- **SynthRAD2023:** public challenge dataset, paired MRI + CT, 1080 patients across centres.
- **Brain subset:** each .npy file has shape $[2, H, W, D]$:
 - Channel 0: MRI (T1-weighted)
 - Channel 1: CT (in HU)
 - Volume: $\approx 192 \times 192 \times 96$ voxels
- **Normalisation:** both modalities normalised to $[-1, +1]$.
 - CT: clip $[-1024, 1650]$ then linear rescale.
 - MRI: divide by 99th percentile, clip $[0, 1]$, rescale.
- **Patch strategy:** train on $64 \times 64 \times 4$ random patches; sliding-window inference at test time.



Why Diffusion Models?

- **GANs:** adversarial training, sharp outputs but unstable. Mode collapse and HU inaccuracy at high-contrast interfaces (bone, air).
- **VAEs:** stable training but blurry outputs — bone edges smoothed out.
- **Diffusion Models (chosen):**
 - Stable maximum-likelihood training (no adversarial).
 - Sharp outputs comparable to GANs.
 - Calibrated uncertainty via learned variance.
 - Iterative denoising allows MRI conditioning at every step.
- **Specific variant:** IDDPM (Nichol & Dhariwal) with `predict_xstart=True`, `learn_sigma=True`, 1000 train / 50 inference steps.



Baseline — DDPM with Swin Transformer

Baseline Overview

- **Architecture:** 3D UNet with Swin Transformer blocks (**SwinVITModel**) inside a DDPM framework.
- **Conditioning:** MRI is concatenated with noisy CT x_t at the input — model sees MRI at every denoising step.
- **Prediction target:** the network directly predicts the clean CT $\hat{x}_0$ instead of noise ϵ — enforces HU accuracy at every training step.
- **Output:** 2-channel volume — $\hat{x}_0$ (predicted CT) + v (variance logit), thanks to `learn_sigma=True`.
- **Plan for next slides:** first the complete pipeline, then dive inside the U-Net and its building blocks.

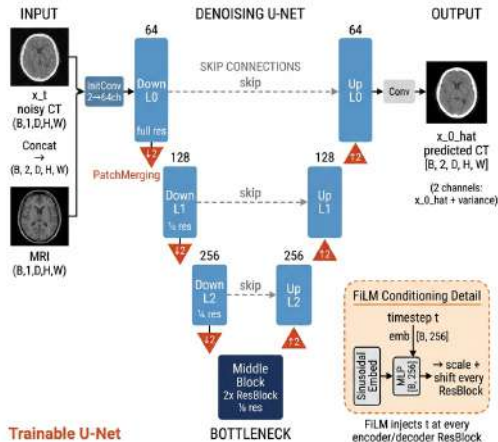


Baseline Pipeline — DDPM + Swin (Complete View)

What's happening:

- MRI concat with noisy CT x_t at input — the only conditioning channel.
- U-Net iteratively denoises across 50 timesteps.
- FiLM injects timestep t into every ResBlock $\rightarrow$ same weights handle different noise levels.
- Output is 2-channel: $\hat{x}_0$ (predicted CT) + v (variance) — enables IDDPM's calibrated sampling.
- **Why this design?** Stable training (vs GANs), sharp output (vs VAEs), uncertainty-aware.

(a) Baseline Diffusion Model – DDPM + Swin

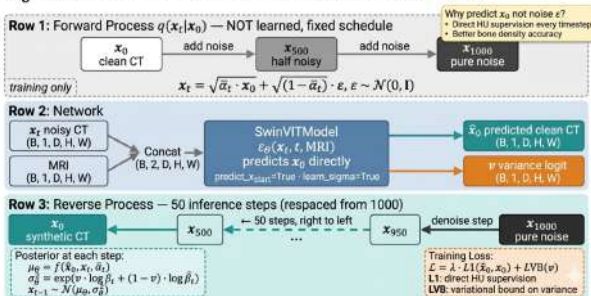


x_0 Prediction — How the Network Denoises CT

Why predict x_0 , not ϵ ?

- Standard DDPM predicts noise ϵ . We instead predict the clean CT $\hat{x}_0$.
- **Direct HU supervision** at every timestep — L1 loss on $\hat{x}_0$ vs ground-truth CT enforces calibrated Hounsfield Units at each step.
- Predicting ϵ would only supervise via the inverse mapping, weaker signal at high t .
- **Better bone density accuracy** — bone (HU > 300) is rare; direct x_0 supervision avoids smoothing it out.
- Variance v learned jointly $\rightarrow$ posterior $\mathcal{N}(\mu_\theta, \sigma_\theta^2)$ for sampling.

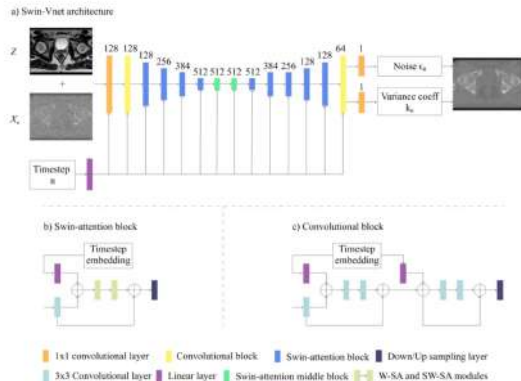
x_0 Prediction — How SwinViTModel Denoises CT



Inside the U-Net — SwinViTModel Architecture

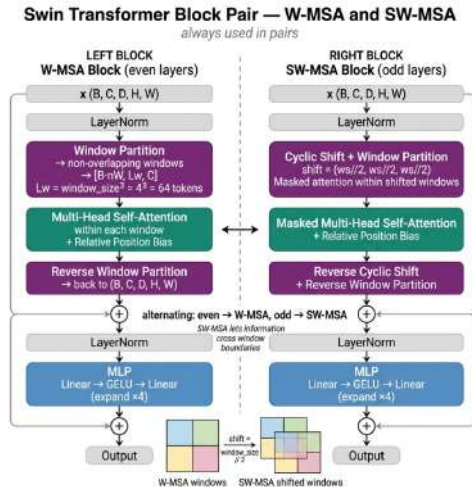
Architecture choices:

- 4-level 3D U-Net: $64 \rightarrow 128 \rightarrow 256$ channels in encoder, mirrored decoder.
- ResBlocks fused with Swin Transformer blocks at each level $\rightarrow$ local + global features.
- PatchMerging downsampling ($\downarrow 2$) preserves anisotropic spacing.
- Trilinear upsampling avoids checkerboard artifacts.
- Skip connections at every level for fine detail.
- **Why U-Net?** Dense voxel prediction needs both encoder context and decoder spatial precision.



Inside the Swin Block — W-MSA + SW-MSA Pair

- Standard MSA is $\mathcal{O}(N^2)$ — infeasible on 3D volumes.
- **W-MSA**: attention within local windows (e.g. $4 \times 4 \times 4$).
- **SW-MSA**: cyclic shift before partition — lets information cross window boundaries.
- Used in alternating pairs: even layer $\rightarrow$ W-MSA, odd $\rightarrow$ SW-MSA.
- Pre-norm architecture: LayerNorm $\rightarrow$ Attention $\rightarrow$ residual $\rightarrow$ LayerNorm $\rightarrow$ MLP $\rightarrow$ residual.

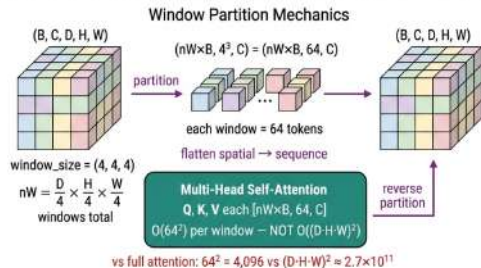


Inside the Attention — 3D Window Partition + RPB

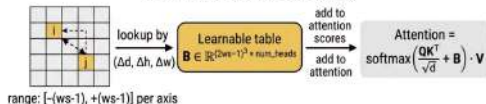
Why window-based attention?

- Full attention on a $32 \times 128 \times 128$ patch is $O(L^2) \approx 10^{11}$ entries — infeasible.
- Partition into $4 \times 4 \times 4 = 64$ -token windows: $64^2 = 4096$ per window. Linear in volume size.
- **Relative Positional Bias (RPB)**: learnable table $B \in \mathbb{R}^{(2ws-1)^3 \times \text{heads}}$ encodes inter-token distances.
- Better than absolute positions — RPB transfers across volume sizes.
- Attention = $\text{softmax}(QK^\top / \sqrt{d} + B) \cdot V$.

3D Window Partition & Relative Positional Bias



Relative Positional Bias (RPB)



RPB encodes which window positions are near each other.
Not absolute positions — transfers across different volume sizes.



Pretrained MRI Encoder & Hybrid Model

Why a Pretrained MRI Encoder? (Our Design)

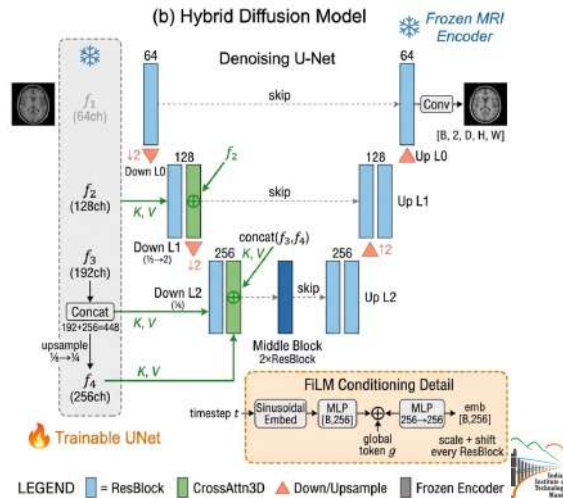
- **Limitation of baseline:** MRI is just concatenated at the input. The U-Net has to learn anatomical features from scratch through noise.
- **Our design:** pretrain a 3D MRI semantic encoder with a custom dual-task objective, then *freeze* it and inject its features into the diffusion U-Net.
- **Dual-task pretraining (our combination):**
 - Head A: MRI reconstruction (preserves structure).
 - Head B: CT prediction (learns HU-discriminative features).
 - Loss: $0.5 \cdot L_1(\text{CT}) + 0.3 \cdot L_1(\text{MRI}) + 0.2 \cdot \text{SSIM}(\text{MRI})$.
- **Two injection pathways (our configuration):**
 - Multi-scale MRI features $\rightarrow$ **cross-attention (K, V)** at matching denoiser resolutions ($f_2 \rightarrow$ Down L1, $f_3 \parallel f_4 \rightarrow$ Down L2).
 - Global MRI token added to timestep embedding $\rightarrow$ **FiLM scale+shift** every ResBlock.



Hybrid Diffusion Pipeline (Complete View)

Why f_1 skipped, $f_3 \parallel f_4$ concatenated?

- **No attention on f_1 (full res):** cross-attn cost is $\mathcal{O}(N \cdot M)$ where N, M are token counts. At full res $N \approx D \cdot H \cdot W$ — prohibitively expensive.
- Also f_1 carries low-level edge features; the U-Net's own ResBlocks recover these locally. Attention adds no value here.
- **$f_3 \parallel f_4$ concat at Down L2:** f_3 is 192ch at $\frac{1}{4}$ res, f_4 is 256ch at $\frac{1}{8}$ res. Trilinearly upsample f_4 , concat along channels $\rightarrow$ 448ch context.
- **Why concat, not 2 separate cross-attns?** Single cross-attn with combined K/V is cheaper. f_4 provides global anatomy summary, f_3 provides local structure.
- **Two pathways:** $f_2 \rightarrow$ K/V at L1 (128ch); $f_3 \parallel f_4 \rightarrow$ K/V at L2 (448ch); global token $\rightarrow$ FiLM every ResBlock.

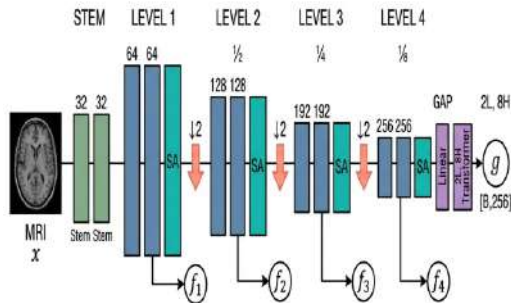


Inside the Encoder — MRI Semantic Encoder Architecture

Design choices:

- 4-level 3D conv + window self-attention encoder.
- Channels: $1 \rightarrow 32 \rightarrow 64 \rightarrow 128 \rightarrow 192 \rightarrow 256$.
- Anisotropic stride (2, 2, 1) — preserves thin-slice depth dim.
- GroupNorm (not BatchNorm) — stable at batch size 2.
- Window-attention with RPB at every level $\rightarrow$ spatial anatomy relationships.
- Global token from AdaptiveAvgPool $\rightarrow$ 2-layer Transformer bottleneck $\rightarrow$ summary $g \in \mathbb{R}^{256}$.
- **Outputs:** 4 multi-scale feature maps + 1 global token.

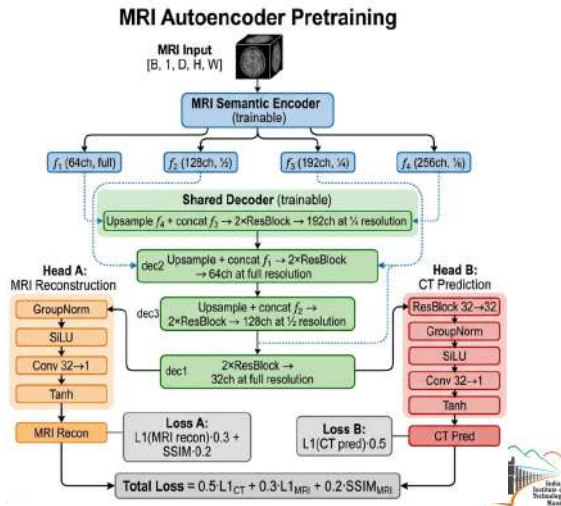
(a) MRI Semantic Encoder



Inside the Pretraining — MRI Autoencoder Dual-Task

Why dual-task pretraining?

- **Head A (MRI recon):** forces structural completeness — encoder must preserve anatomy.
- **Head B (CT pred):** forces CT-discriminative features — bone density, tissue boundaries, HU-relevant cues.
- Loss:
 $0.5 L_1(\text{CT}) + 0.3 L_1(\text{MRI}) + 0.2 \text{SSIM}(\text{MRI})$.
- **Single task is insufficient:** MRI-only loses CT-relevant cues; CT-only loses MRI-specific anatomy.
- After pretraining, decoder heads are **discarded** — only encoder weights are kept and frozen.
- Inspired by Tang et al. self-supervised Swin pretraining.



Future Work — Diffusion + Pretrained Encoder

- 1 **Dose-metric loss term.** *Why:* SynthRAD report — MAE/SSIM don't correlate with dose accuracy. Optimising image metrics misses clinical target. *What:* differentiable γ -pass-rate or dose-difference penalty alongside L1+VB.
- 2 **Per-voxel uncertainty maps.** *Why:* already run 3 MC samples but throw away variance. Bone/air boundaries are exactly where clinicians need uncertainty. *What:* output per-voxel variance map. Zero extra compute.
- 3 **DDIM deterministic sampling.** *Why:* 3 MC runs needed only because sampling is stochastic. Determinism removes run-to-run variance. *What:* ODE integration instead of stochastic sampling. Same weights, $3\times$ faster.
- 4 **Swin UNETR pretrained init $\rightarrow$ dual-task fine-tune.** *Why:* paired SynthRAD set is small (1080 patients). Tang et al. pretrained on 5050 CTs — richer prior. *What:* init MRISemanticEncoder from Swin UNETR, then dual-task fine-tune.
- 5 **Multi-contrast MRI conditioning.** *Why:* bone is dark in both T1 and T2 — single contrast can't disambiguate skull vs CSF. *What:* extend input to T1+T2+FLAIR. Dual-task learns which contrast matters per tissue.



The Mamba Solution

What is Mamba & Why Do We Need It?

- **The Challenge:** 3D Transformers suffer from quadratic $\mathcal{O}(N^2)$ attention cost. Window attention helps but still bounds the receptive field.
- **The Mamba Advantage (Gu & Dao):**
 - Selective State Space Model (SSM): represents the volume as a 1D token sequence.
 - Linear $\mathcal{O}(N)$ complexity in sequence length.
 - Long-range context via accumulated hidden state.
- **Trade-off:** flattening 3D into 1D destroys spatial structure — different scanning strategies (axis-wise, tri-planar) try to mitigate this.
- **We evaluate 4 deterministic Mamba architectures** as parallel approaches to the diffusion baseline.



Deterministic Mamba Architectures

1. SegMamba — Hybrid CNN + Mamba U-Net

Why move here? Swin baseline restricts attention to 4^3 windows — global anatomy (skull shape, ventricles) outside any window is invisible. Mamba SSM gives *linear-cost* global context.

Intuition: treat volume as 1D sequence. SSM accumulates hidden state summarising everything seen → every voxel “remembers” the whole patch.

Architecture:

- 4-stage symmetric U-Net (easy ablation vs Swin).
- Each block: *ResConv* (local) + *MambaBlock3D* (global).
- 5D volume flattened to 1D sequence, scanned, reshaped back.

Result & lesson: soft-tissue MAE drops to 38.15 HU ✓, but Bone MAE 208.52 HU — flattening order is arbitrary, breaks spatial structure → next idea.

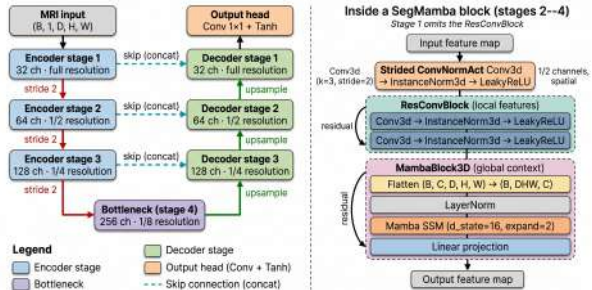


SegMamba Architecture

Key observations:

- Hybrid: ResConv (local) + MambaBlock3D (global) at each level.
- Volume flattened to a single 1D sequence for Mamba scan.
- **Strength:** excellent soft-tissue MAE (38.15 HU).
- **Weakness:** flattening order destroys spatial structure $\rightarrow$ poor bone boundary (208.52 HU).
- Motivates moving to axis-aware scanning (next).

SegMamba: Architecture Overview



2. U-Mamba — Tighter Conv-Mamba Coupling

Why move here?

- SegMamba alternates Conv block $\rightarrow$ Mamba block. Two separate stages $\rightarrow$ local features computed first, global mixing only afterwards. Information flow is sequential, not synergistic.
- Also, do we even need iterative diffusion? If a deterministic single-pass network can match diffusion quality, inference is $50\times$ faster.

Intuition: fuse Conv and Mamba inside one block — local kernels and global SSM both touch the same feature tensor. “Refine globally as you go”, not “go locally then globally”.

Architecture:

- **UMambaBlock:** Conv $\rightarrow$ InstanceNorm3d $\rightarrow$ Mamba, all stacked tightly.
- Deterministic forward pass — no diffusion, no MC sampling.
- Compound staged loss handles bone class imbalance:
 - Phase 1 (warm): wMAE only — bone weight $\times 3$ to fix HU calibration first.
 - Phase 2 (sharpen): add 3D SSIM + AFP loss after epoch 100.

Result & lesson: PSNR 25.23 dB — best deterministic Mamba so far. But still uses 1D flattening $\rightarrow$ structural bias remains $\rightarrow$ next idea: scan along axes, not flatten.

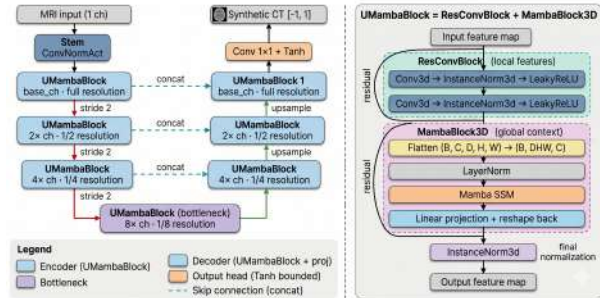


U-Mamba Architecture

Tighter coupling:

- $\text{UMambaBlock} = \text{Conv} \rightarrow \text{InstanceNorm} \rightarrow \text{Mamba}$ directly at each stage.
- *No diffusion* — single-pass deterministic regression to CT.
- Compound staged loss: HU-aware MAE first, SSIM + AFP loss after epoch 100.
- Bone weighted $\times 3$ in MAE to compensate for class imbalance.
- **Trade-off:** faster (no 50-step diffusion) but loses uncertainty calibration.
- Best PSNR among deterministic Mambas: 25.23 dB.

U-Mamba: Architecture Overview



Architecture Comparison: UMamba vs. SegMamba

UMamba

- **Entry Point:** Explicit Stem (ConvNormAct)
- **Encoder Stage:** Unified UMambaBlock + InstanceNorm3d
- **Downsampling:** Independent ConvNormAct layers
- **Decoder:** UMambaBlock $\rightarrow$ proj

SegMamba

- **Entry Point:** Internal Expansion (enc1_conv in Stage 1)
- **Encoder Stage:** Sequential ResConvBlock $\rightarrow$ MambaBlock3D
- **Downsampling:** Strided Conv starts each stage
- **Decoder:** ResConvBlock $\rightarrow$ MambaBlock3D $\rightarrow$ proj



3. TriAxial Mamba — Axis-Aware Scanning

Why move here? SegMamba/U-Mamba flatten (D, H, W) to one 1D sequence. Adjacent voxels become far apart in sequence. Bone is sparse (4%) but structured (skull surface) $\rightarrow$ flatten order kills bone reconstruction.

Intuition: brain has three meaningful axes. Run *three separate* 1D scans (one per axis) so each voxel meets its anatomical neighbours along natural axis ordering.

Mechanics:

- D-axis (Sup-Inf), H-axis (Ant-Post), W-axis (Left-Right) — bidirectional per axis.
- 6 SSMs total. Sequence length is D , H , or W — much shorter than $D \cdot H \cdot W$.
- Fusion: concat 3 outputs along channels, $1 \times 1 \times 1$ Conv3d back to C .

Result & lesson: Bone MAE drops to 196 HU. PSNR 25.71 dB. Still missing: skip-feature gating, deep supervision, phased loss $\rightarrow$ TriMamba-UNet V2.



4. TriMamba-UNet V2 — Full Architecture

Why move here? TriAxial fixes scan order but two problems remain: (1) skip connections pass *all* encoder features (noisy/irrelevant included); (2) deep decoder layers suffer gradient vanishing. Plus: earlier hard loss switch caused NaN at epoch 52 (LR restart collision).

Intuition: keep TriAxial as global-context engine, add three engineering fixes:

- 1 Filter skip features → **CBAM3D** (channel + spatial attention).
- 2 Gradient signal at intermediate decoder levels → **deep supervision** (aux heads at Dec2, Dec3).
- 3 Avoid gradient shocks → **CompoundLossV2 staged ramp** (linear ramp 30 epochs/loss).

Result: best raw image quality — PSNR 25.79 dB, SSIM 0.856, Bone MAE 189 HU.

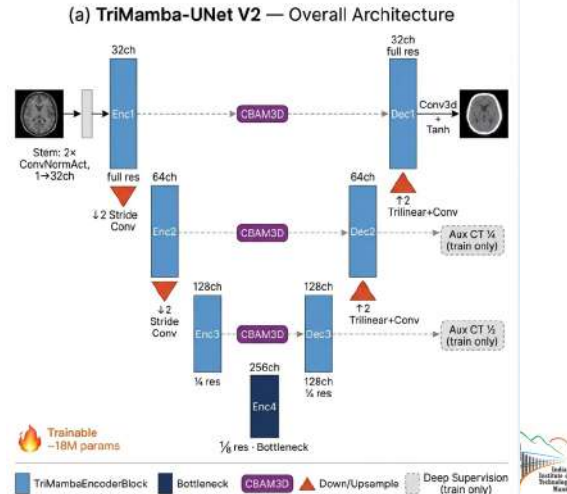
Next slides: full architecture, then dive into each component.



TriMamba-UNet V2 — Complete Architecture

Why this combination?

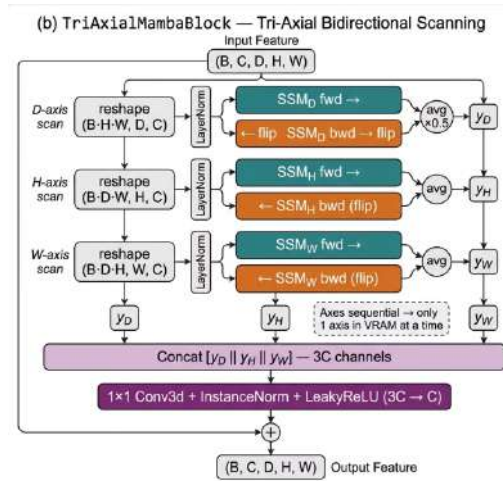
- TriMambaEncoderBlock at every encoder/decoder level → axis-aware global context throughout.
- CBAM3D on skip connections → filters encoder features before decoder fusion.
- Deep supervision at Dec2, Dec3 → prevents gradient vanishing in deep decoder.
- Trilinear upsampling → no checkerboard.
- Gradient checkpointing scoped to Mamba only → fits 24GB GPU.
- **Best raw image quality:** PSNR 25.79 dB, SSIM 0.856.



Inside TriMamba — TriAxial Bidirectional Scan Block

6 SSMs per block:

- One forward + one backward SSM per axis (D, H, W) $\rightarrow$ 6 total.
- **Why bidirectional?** Causal SSM only sees past tokens; backward scan recovers future context.
- **Why per-axis, not flatten?** Flatten destroys structure and creates length $D \times H \times W$ sequences. Per-axis keeps short sequences.
- **Sequential processing** of axes: only 1 axis's intermediates in VRAM at a time.
- Fusion: $1 \times 1 \times 1$ Conv3d ($3C \rightarrow C$) + residual.



6 Mamba SSM blocks total $\cdot d_{\text{state}} = 16 \cdot$ axes processed sequentially

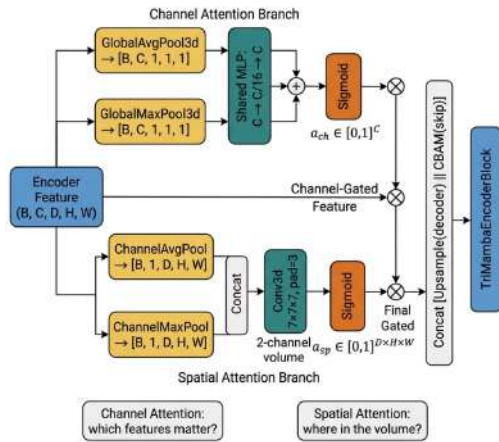


Inside the Skip Connections — CBAM3D Attention Gate

Why CBAM3D, why on skips?

- **Standard skip connections** pass encoder features blindly — decoder gets all channels regardless of relevance. At noisy levels and ambiguous brain regions, this misguides the decoder.
- **Why not SE block?** SE only does channel attention — misses spatial selectivity needed for sparse bone voxels.
- **Why not self-attention?** Quadratic cost on full 3D feature maps; CBAM is two cheap pooling-based modules.
- **Why on skips, not blocks?** Skips carry uncurated encoder features into the decoder — exactly where filtering matters most. Mamba blocks already do their own context mixing.

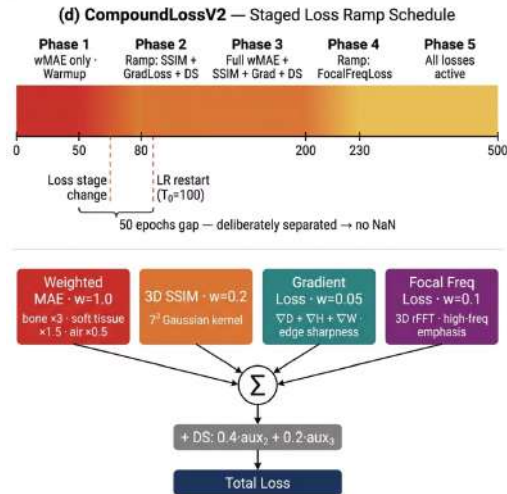
(c) CBAM3D — Attention-Gated Skip Connection



Training Schedule — Why Phased Loss Ramp?

Why different losses for different phases?

- **Phase 1 (warmup, wMAE only):** model first learns the coarse HU distribution. Adding SSIM/Grad early would conflict with intensity calibration.
- **Phase 2 (ramp SSIM + Grad + DS):** once HU is calibrated, structural sharpening kicks in. Linear ramp prevents gradient shock.
- **Phase 3 (ramp FFL):** Focal Frequency Loss applies only after structure is stable — targets high-frequency bone edges that L1/SSIM smooth out.
- **Why phased?** Hard switch at fixed epoch caused NaN at epoch 52 (LR restart collision). Linear ramp eliminates this.
- **Why FFL specifically?** L1/SSIM are biased toward low-frequency consensus — bone edges blur. FFL adaptively up-weights frequencies where error is large.



5. TriPlane Mamba — Why Planes Instead of Axes?

What changes from TriAxial? TriAxial scans three independent 1D sequences (D, H, W axes) — each voxel only meets its neighbours along one axis at a time.

Core Insight (Wang et al., MICCAI 2024): scan along the three 2D *planes* (axial hw, coronal dw, sagittal dh) instead. Each plane's Mamba block processes 2D spatial context in one pass leads to richer structural co-dependencies captured per scan.

Two problems with axis-only scanning:

- **Missing intra-plane structure:** a D-axis scan sees skull top-to-bottom but misses lateral skull curvature within each axial slice.
- **No multi-scale local context:** SSM global branch needs a complementary local branch to anchor fine bone/tissue boundaries.

Triplanar mamba adds two innovations on top of TriAxial:

- 1 **Tri-planar bidirectional SSM:** three Mamba blocks scan hw-, dw-, dh-planes independently, outputs summed.
- 2 **Multi-scale dilated local branch:** four parallel 3D convolutions (dilation $d = 1, 2, 4, 8$) capture local texture at multiple receptive fields simultaneously.

Retained from TriMamba-UNet V2: CBAM3D-gated skip connections, deep supervision, CompoundLossV2 staged ramp.



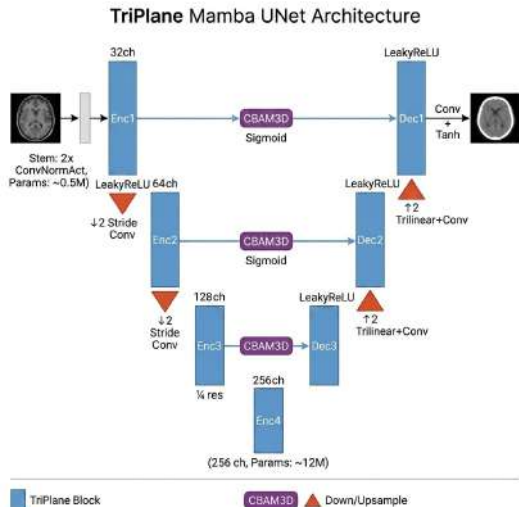
TriPlane Mamba Block — Dual-Branch Design

Local Branch (blue):

- $4 \times$ parallel 3D Conv with dilations $d \in \{1, 2, 4, 8\} + \text{LeakyReLU}$.
- Multi-scale receptive field without increasing sequence length.
- Captures fine bone edges and air-tissue interfaces locally.
- No global mixing — complements, not competes with, SSM branch.

Global Branch (red):

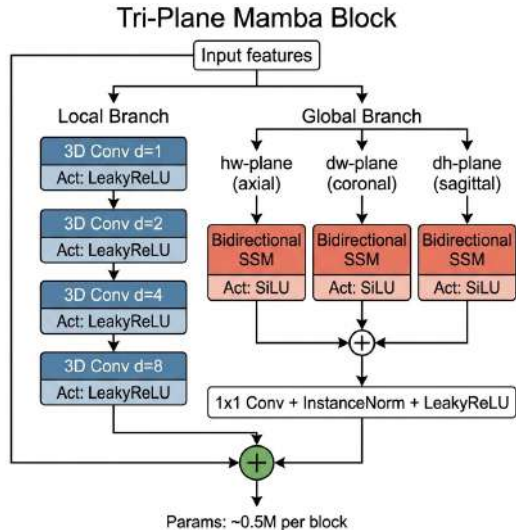
- Three *independent* Bidirectional SSMs on hw- (axial), dw- (coronal), dh- (sagittal) planes.
- Each SSM sees a full 2D plane slice sequence — richer spatial context than a 1D axis scan.
- Outputs element-wise summed $\rightarrow 1 \times 1$ Conv + InstanceNorm + LeakyReLU.



TriPlane Mamba UNet — Full Architecture

Architecture choices:

- Stem: $2 \times \text{ConvNormAct}$ ($\approx 0.5\text{M}$ params) for initial feature extraction.
- 4-level UNet: $32 \rightarrow 64 \rightarrow 128 \rightarrow 256$ channels; $\downarrow 2$ stride conv at each level.
- **Every encoder/decoder block is a TriPlane Block** : local-global fusion throughout the hierarchy.
- Bottleneck Enc4 (256ch, $\frac{1}{4}$ res): $\approx 12\text{M}$ params is the bulk of global context.
- **CBAM3D + Sigmoid** on all skip connections — gates irrelevant encoder features before decoder fusion (inherited from V2).
- Trilinear+Conv upsampling leads to no checkerboard artifacts.
- Output: Conv + Tanh $\rightarrow$ normalised HU



Future Work — Mamba Architectures

- 1 **Tri-axial + global attention at bottleneck.** *Why:* Mamba's causal scan misses cross-axis dependencies. At $\frac{1}{8}$ res, global attention is cheap. *What:* one global attention block between Enc4 and Dec3 (nnFormer interleaving idea).
- 2 **Curriculum learning on tissue difficulty.** *Why:* bone is $\sim 4\%$ of voxels but $\sim 50\%$ of total error. Random sampling under-trains bone. *What:* start with soft-tissue-heavy patches, progressively add bone-heavy patches.
- 3 **Physics-informed HU constraint.** *Why:* HU encodes electron density: $\rho_e = k(\text{HU} + 1000)$. Network can predict physically impossible HU. *What:* encoder identifies skull $\rightarrow$ region-specific HU range penalty (skull HU > 200).
- 4 **CBAM3D ablation studies.** *Why:* we claim CBAM3D drives gains, but channel-only or spatial-only might suffice cheaper. *What:* $\{\text{channel, spatial, full}\} \times \{\text{skips, inside blocks}\}$ grid.
- 5 **Mamba denoiser + frozen pretrained encoder ablation.** *Why:* Mamba's scan may interact differently with injected MRI than Swin's attention. Open question. *What:* inject our dual-task encoder into TriMamba via cross-attention. Direct comparison vs Swin hybrid.
- 6 **Ablation: planar vs. axis-wise scanning on bone reconstruction.** *Why:* TriPlane and TriAxial differ only in the scan module, enabling a controlled comparison of 2D planar vs 1D axial context for Bone MAE. *What:* Replace TriPlane SSMs with TriAxial SSMs, retrain with the same hyperparameters, and compare Bone MAE and Gamma pass rates.
- 7 **Dilation schedule search for the local branch.** *Why:* dilations $\{1, 2, 4, 8\}$ were inherited from TP-Mamba's segmentation setting. Optimal receptive fields for HU regression on brain volumes may differ. *What:* grid search over $\{1, 2, 4, 8\}$, $\{1, 3, 6, 12\}$, and $\{1, 2, 4\}$ (3-branch) configurations. Measure Bone MAE and parameter cost.
- 8 **Learnable plane-wise fusion weights.** *Why:* axial, coronal, and sagittal planes contribute unequally to bone reconstruction — skull is thin in coronal but wide in axial. Uniform summing ignores this. *What:* replace element-wise sum of three SSM outputs with a learned channel-wise softmax weighting. Minimal parameter overhead ($3 \times C$ scalars per block).



Diffusion UMamba — DDPM with Mamba

- **Core Idea:** DDPM framework but replace W-MSA / SW-MSA layers in the denoiser with **Bidirectional Mamba SSM** blocks.
- **Key Advantages:**
 - Linear $\mathcal{O}(N)$ complexity allows much larger training patches ($128 \times 128 \times 16$ vs $64 \times 64 \times 4$ for Swin baseline).
 - Theoretically infinite receptive field — entire patch attended without window shifting overhead.
- **Clinical Reality vs Theory:**
 - Raw Diffusion UMamba initially trailed Swin baseline in PSNR / SSIM.
 - With tissue-weighted losses, deep supervision, and timestep respacing, surpassed baseline on **dosimetric** metrics.
 - Achieves **99.55% Gamma 2%/2mm pass rate** — best clinical accuracy.



Future Work — Diffusion UMamba & GAN Hybrids

- 1 **Latent diffusion (voxel $\rightarrow$ latent).** *Why:* diffusion runs 50 steps on raw voxel patches. Compress volume first $\rightarrow$ each step cheaper. *What:* 3D VAE (MRI+CT jointly), run IDDPM in latent space. Patch shrinks 4–8 $\times$, removes MC bottleneck.
- 2 **GAN-style adversarial loss inside diffusion training.** *Why:* L1+VB encourages low-frequency consensus $\rightarrow$ bone edges blur. GANs recover sharpness. *What:* lightweight 3D discriminator alongside L1+VB. Adversarial term targets high-frequency residual.
- 3 **Bidirectional synthesis (CT $\leftrightarrow$ MRI).** *Why:* our dual-task encoder *already* has both decoder heads. Free bidirectional capability unused. *What:* train both diffusion models (MRI $\rightarrow$ CT, CT $\rightarrow$ MRI) on same frozen encoder. Built-in consistency.
- 4 **2 $\times$ 2 freezing ablation.** *Why:* we freeze encoder, CoCoLIT freezes diffusion. Which drives the gain? Currently confounded. *What:* {frozen-enc, trainable-enc} $\times$ {frozen-diff, trainable-diff} grid.
- 5 **Cycle-consistency from UNIT into diffusion.** *Why:* UNIT cycle loss handles HU calibration at unpaired interfaces; diffusion alone fails on bone-air. *What:* add cycle regulariser to diffusion loss: $f_{\text{CT} \rightarrow \text{MRI}}(f_{\text{MRI} \rightarrow \text{CT}}(x)) = x$.



Comparison — GAN Approaches

Other Approaches — Supervised & Unsupervised GANs

Pix2Pix GAN (Supervised):

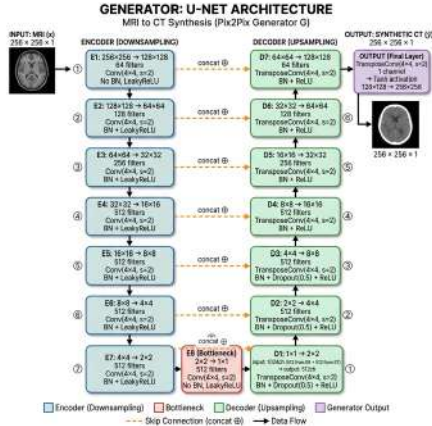
- Paired voxel-wise L1 loss + adversarial loss.
- Strong soft-tissue reconstruction, but fails on bone and air boundaries (> 600 HU error).
- Gamma 1%/1mm pass rate only 27.33%.

UNIT GAN (Unsupervised):

- Shared latent space between MRI and CT, cycle consistency.
- No voxel-level ground-truth supervision.
- Plausible distributions but massive HU inaccuracy at high-contrast interfaces.



Pix2Pix GAN Architecture

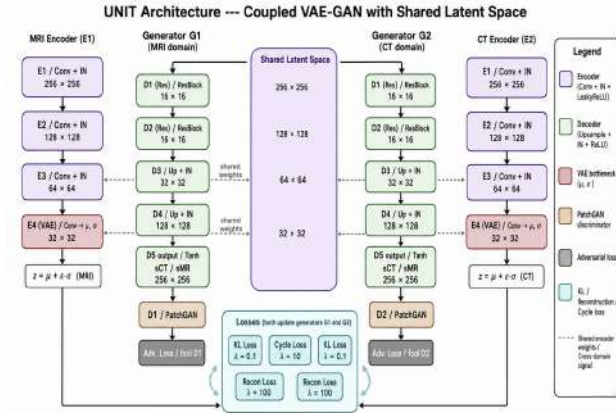


Key Components:

- **Generator:** U-Net mapping MRI ($256 \times 256 \times 1$) directly to synthetic CT.
- **Encoder:** 7 downsampling stages + 1 bottleneck. Uses Conv(4x4, s=2), LeakyReLU, and BN (except in E1 and E8).
- **Decoder:** 7 upsampling stages. Uses TransposeConv(4x4, s=2), BN, and ReLU. Dropout(0.5) is applied in D1–D3.
- **Output:** Final layer uses TransposeConv(4x4, s=2) and Tanh activation to output the normalised CT.



UNIT GAN Architecture



Key Components:

- **Coupled VAE-GAN:** Encoders map MRI/CT to a shared latent space using the reparameterisation trick.
- **Architecture:** Encoders use Conv + IN + LeakyReLU; Generators use ResBlocks followed by Upsample + IN + ReLU.
- **Discriminators:** PatchGAN discriminators (D1, D2) provide the adversarial loss for each domain.
- **Losses:** Jointly optimised with Adversarial loss, Reconstruction loss ($\lambda = 100$), Cycle loss ($\lambda = 10$), and KL loss ($\lambda = 0.1$).



Comprehensive Results

Comprehensive Dosimetric & Quantitative Results

Model	MAE (↓)	PSNR (↑)	SSIM (↑)	Bone MAE (↓)	γ 1%/1mm (↑)	γ 2%/2mm (↑)
Baseline (Swin DDPM)	–	21.09 dB	0.720	335.79 HU	88.74%	98.51%
SegMamba	0.0480	24.79 dB	0.8432	208.52 HU	91.61%	99.35%
U-Mamba	0.0443	25.23 dB	0.8531	–	–	–
TriAxial Mamba	0.0458	25.71 dB	0.8483	196.20 HU	88.71%	98.83%
TriMamba-UNet V2	0.0445	25.79 dB	0.8561	189.39 HU	90.61%	99.14%
Diffusion UMamba	–	22.49 dB	0.768	192.50 HU	93.26%	99.55%
Pix2Pix	–	20.98 dB	0.8403	684.58 HU	27.33 %	33.61%
Unit GAN:	–	19.06 dB	0.8137	705.09 HU	26.14 %	31.56%

- **Image Quality vs. Dosimetry: TriMamba-UNet V2** wins raw image metrics (PSNR, SSIM, Bone MAE).
- **Clinical Superiority: Diffusion UMamba** dominates dosimetric metrics — highest Gamma pass rates — most viable for radiotherapy.

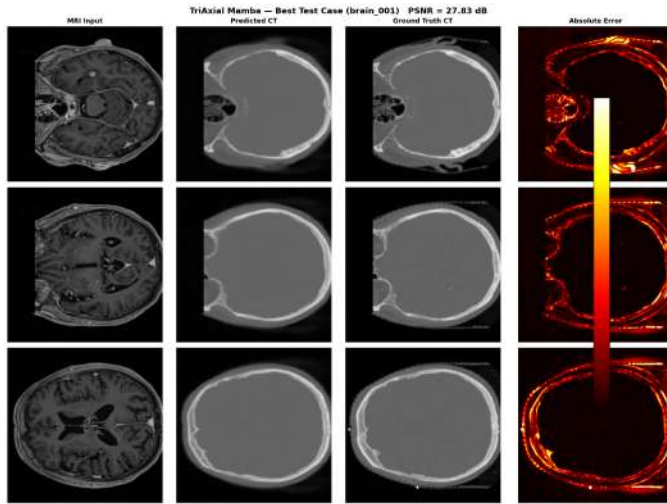


PSNR Results and Analysis

Model	PSNR(1D)	PSNR (2D)	PSNR(3D)
Baseline (Swin DDPM)	28.48 dB	21.67 dB	21.09 dB
SegMamba	32.84 dB	25.42 dB dB	24.79 dB
U-Mamba	33.88 dB	25.78 dB	25.23 dB
TriAxial Mamba	33.32 dB	26.32 dB	25.71 dB
TriAxial Mamba V2	33.77 dB	26.40 dB	25.79 dB
Diffusion UMamba	27.50 dB	24.39 dB	22.49 dB
Pix2Pix	33.49 dB	31.37 dB	20.98 dB
Unit GAN	31.83 dB	29.47 dB	19.06dB

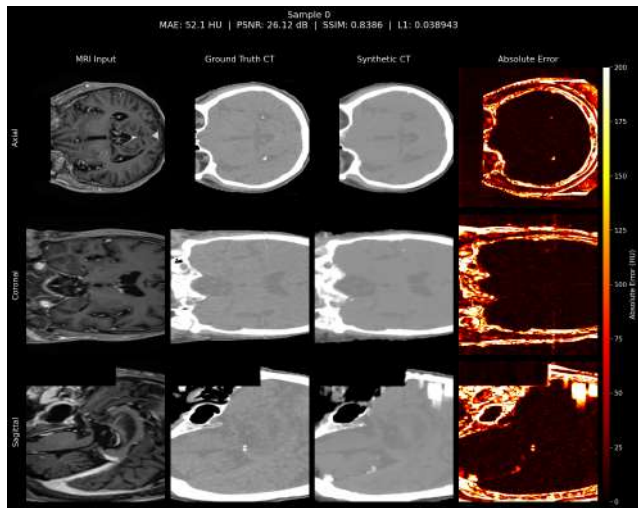


Visual Results — TriAxial Mamba (best test case)



MRI input · Predicted CT · Ground Truth · Absolute Error

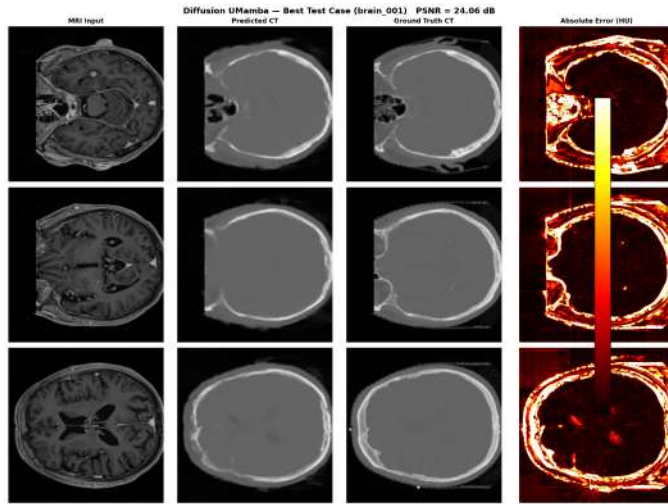
Visual Results — TriMamba-UNet V2 (Pretrained Encoder + Hybrid)



MRI input · Predicted CT · Ground Truth · Absolute Error



Visual Results — Diffusion UMamba (best dosimetric)

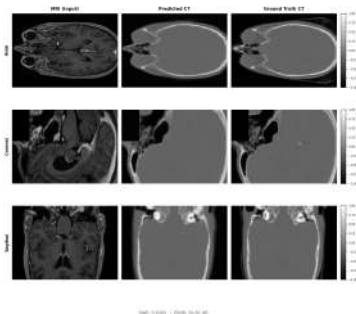


MRI input · Predicted CT · Ground Truth · Absolute Error

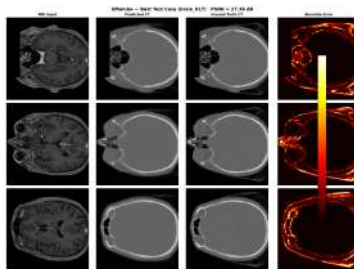
Visual Comparison — SegMamba vs U-Mamba vs TriPlane

SegMamba

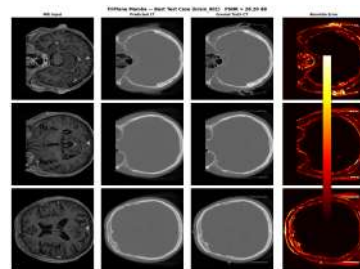
Case: brain_017



U-Mamba



TriPlane Mamba



MRI input · Predicted CT · Ground Truth · Absolute Error



Diffusion & Backbone:

- 1 Ho et al. — DDPM, NeurIPS (arXiv:2006.11239)
- 2 Nichol & Dhariwal — IDDPM, ICML (arXiv:2102.09672)
- 3 Liu et al. — Swin Transformer, ICCV (arXiv:2103.14030)
- 4 Hatamizadeh et al. — Swin UNETR, MICCAI (arXiv:2201.01266)
- 5 Ronneberger et al. — U-Net, MICCAI (arXiv:1505.04597)
- 6 Çiçek et al. — 3D U-Net, MICCAI (arXiv:1606.06650)

MRI-to-CT Synthesis:

- 7 Pan et al. — MC-DDPM (Swin-Vnet for MRI-to-CT), Med Phys (arXiv:2305.19467)
- 8 Lyu & Wang — CT-MRI diffusion benchmarking (arXiv:2209.12104)
- 9 Dalmaz et al. — ResViT (cross-modal synthesis), IEEE TMI (arXiv:2106.16031)
- 10 Tang et al. — Self-supervised Swin 3D (motivates dual-task pretraining), CVPR (arXiv:2111.14791)

Conditioning & Injection:

- 11 Zhang et al. — ControlNet (zero-conv injection), ICCV (arXiv:2302.05543)
- 12 Perez et al. — FiLM, AAAI (arXiv:1709.07871)

Mamba & SSM:



References II

- 13 Gu & Dao — Mamba, NeurIPS (arXiv:2312.00752)
- 14 Gu et al. — HiPPO, NeurIPS (arXiv:2008.07669)
- 15 Zhu et al. — Vision Mamba (bidirectional scan), ICML (arXiv:2401.09417)
- 16 Liu et al. — VMamba (cross-scan), NeurIPS (arXiv:2401.10166)
- 17 Xing et al. — SegMamba (3D medical Mamba), MICCAI (arXiv:2401.13560)
- 18 Ma et al. — U-Mamba (arXiv:2401.04722)
- 19 Wang et al. — Tri-Plane Mamba, MICCAI (arXiv:2409.08492)

Attention & Loss:

- 20 Woo et al. — CBAM, ECCV (arXiv:1807.06521)
- 21 Jiang et al. — Focal Frequency Loss, ICCV (arXiv:2012.12821)

Dataset:

- 22 Thummerer et al. — SynthRAD2023 dataset, Med Phys (arXiv:2303.16320)
- 23 Lombardo et al. — SynthRAD2023 challenge report, MIA (arXiv:2403.08447)



Thank You!

Questions are welcome.

Code & Models:

`github.com/anishk85/MRI-TO-SCT-mamba-diffusion-gans-`

Appendix — Technical Details

Derivations, equations, expert Q&A

A0. Parameter Counts — All Models

ID	Model	Parameters
1	SegMamba	13,324,353 (13.32 M)
2	U-Mamba	13,381,697 (13.38 M)
3	TriAxial Mamba / TriMamba-UNet V2	19,380,109 (19.38 M)
4	TriPlane Mamba / TriPlaneMamba-UNet	19,902,989 (19.90 M)
5a	Pretrained Encoder Diffusion — Stage 1 (MRI Encoder)	16,039,304 (16.04 M)
5b	Pretrained Encoder Diffusion — Stage 2 (Full Hybrid)	55,137,162 (55.14 M)
5c	Stage 2 trainable (encoder frozen)	39,097,858 (39.10 M)
6	Diffusion UMamba	52,839,234 (52.84 M)

Stage 2 full hybrid = frozen encoder + trainable diffusion network. Trainable count excludes frozen MRI encoder weights.

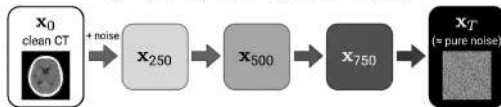


A1. Forward & Reverse Diffusion — Visual Reference

Forward & Reverse Diffusion Process

Forward (fixed schedule, NOT learned)

$$\mathbf{x}_t = \sqrt{\bar{a}_t} \cdot \mathbf{x}_0 + \sqrt{1 - \bar{a}_t} \cdot \epsilon, \epsilon \sim \mathcal{N}(0, \mathbf{I})$$



Closed form: sample $\mathbf{x}_t$ directly from $\mathbf{x}_0$ in $O(1)$ for any t

Reverse (learned by SwinViTModel)

$$p_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \mu_{\theta}(\mathbf{x}_t, t), \Sigma_{\theta}(\mathbf{x}_t, t))$$

Tractable posterior
used for training target:
 $q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\hat{\mu}_t, \hat{\Sigma}_t \mathbf{I})$
Tractable posterior Network learns
to match to match this posterior



50 inference steps (respaced from 1000 training steps)



A2. Forward Process — Closed-Form Derivation

Per-step transition (Markov chain, fixed schedule):

$$q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t \mathbf{I})$$

Closed form to any step t : let $\alpha_t = 1 - \beta_t$, $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$. Then:

$$q(x_t | x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) \mathbf{I})$$

Reparameterization (used for training):

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \mathbf{I})$$

Why this matters: Given x_0 , sample x_t in $\mathcal{O}(1)$ for any $t \in \{1, \dots, T\}$ — no need to iterate the chain forward. Enables minibatch training where every sample uses random t .

Schedule choice: we use *linear* β_t : $\beta_1 = 10^{-4}$, $\beta_T = 0.02$, $T = 1000$. At $t = 1000$: $\bar{\alpha}_T \approx 0$, so $x_T \approx \mathcal{N}(0, \mathbf{I})$ — pure noise.



A3. Reverse Process — Posterior $q(x_{t-1} \mid x_t, x_0)$

Tractable posterior (closed form via Bayes):

$$q(x_{t-1} \mid x_t, x_0) = \mathcal{N}(x_{t-1}; \tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t \mathbf{I})$$

where:

$$\begin{aligned}\tilde{\mu}_t(x_t, x_0) &= \frac{\sqrt{\bar{\alpha}_{t-1}} \beta_t}{1 - \bar{\alpha}_t} x_0 + \frac{\sqrt{\alpha_t} (1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t \\ \tilde{\beta}_t &= \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t\end{aligned}$$

Network learns to approximate this posterior:

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t))$$

x_0 **prediction:** we predict $\hat{x}_0 = f_\theta(x_t, t, \text{MRI})$, then plug into $\tilde{\mu}_t$ formula to get μ_θ . Equivalent to ε -prediction up to algebra:

$$\hat{\varepsilon} = \frac{x_t - \sqrt{\bar{\alpha}_t} \hat{x}_0}{\sqrt{1 - \bar{\alpha}_t}}$$

Why x_0 , not ε ? Direct HU supervision via $L_1(\hat{x}_0, x_0)$ at every t . Better calibration of bone (rare, high-HU) voxels.



A4. Learned Variance — IDDPM learn_sigma=True

Standard DDPM: variance fixed to $\Sigma_\theta = \tilde{\beta}_t \mathbf{I}$ or $\beta_t \mathbf{I}$.

IDDPM (Nichol & Dhariwal): learn variance as interpolation between two extremes:

$$\Sigma_\theta(x_t, t) = \exp(v \cdot \log \beta_t + (1 - v) \cdot \log \tilde{\beta}_t)$$

where $v \in \mathbb{R}^{D \times H \times W}$ is the network's second output channel, $v_i \in [0, 1]$ via output range.

Loss — two terms:

$$\mathcal{L} = \mathcal{L}_{\text{simple}} + \lambda \cdot \mathcal{L}_{\text{VLB}}$$

- $\mathcal{L}_{\text{simple}} = \|\hat{x}_0 - x_0\|_1$ — supervises $\hat{x}_0$ directly. Stop-gradient on v .
- $\mathcal{L}_{\text{VLB}}$ — variational lower bound on $-\log p_\theta(x_0)$, supervises v . Stop-gradient on $\hat{x}_0$.
- $\lambda = 0.001$ — VLB term is loose, scaled small to avoid overpowering L_1 .

Why it helps: smaller variance at early reverse steps (near pure noise) and larger near x_0 — adaptive sampling matches the data manifold curvature. Enables high-quality 50-step inference from 1000-step training.



A5. Timestep Respacing — 1000 train $\rightarrow$ 50 inference

Training: $T = 1000$ steps. Random $t \sim \text{Uniform}\{1, \dots, 1000\}$ per minibatch.

Inference: use 50 evenly-spaced timesteps $\{t_1, \dots, t_{50}\}$ where $t_k = \lfloor (k-1) \cdot \frac{T}{50} \rfloor$. Roughly $\{0, 20, 40, \dots, 980\}$.

Respaced schedule: for selected timesteps $\{t_1, \dots, t_S\}$, recompute:

$$\bar{\alpha}_{t_k}^{\text{new}} = \prod_{s \leq t_k} \alpha_s^{\text{orig}}, \quad \beta_{t_k}^{\text{new}} = 1 - \frac{\bar{\alpha}_{t_k}^{\text{new}}}{\bar{\alpha}_{t_{k-1}}^{\text{new}}}$$

rescale_timesteps: when network was trained with $t \in \{1, \dots, 1000\}$ but inference uses $S = 50$, scale fed timestep:

$$t_{\text{model_input}} = t_k \cdot \frac{1000}{S}$$

— ensures sinusoidal embedding sees the same t values it saw during training.

Why fewer steps work: learned variance v adapts to the larger step size; without learned σ , 50-step inference would suffer.



A6. x_0 -Prediction vs ε -Prediction

Algebraic equivalence:

$$\hat{x}_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \hat{\varepsilon}}{\sqrt{\bar{\alpha}_t}} \iff \hat{\varepsilon} = \frac{x_t - \sqrt{\bar{\alpha}_t} \hat{x}_0}{\sqrt{1 - \bar{\alpha}_t}}$$

But losses differ:

$$\mathcal{L}_\varepsilon = \|\hat{\varepsilon} - \varepsilon\|^2$$

$$\mathcal{L}_{x_0} = \|\hat{x}_0 - x_0\|^2$$

Substituting the equivalence:

$$\|\hat{x}_0 - x_0\|^2 = \frac{1 - \bar{\alpha}_t}{\bar{\alpha}_t} \|\hat{\varepsilon} - \varepsilon\|^2$$

Implication: loss weighting differs across timesteps t . At $t = 1$, $\bar{\alpha}_t \approx 1$, so weight $\rightarrow 0$ (small loss when prediction is easy).

Net effect for medical imaging: ε -prediction wastes capacity on easy timesteps (small noise); x_0 -prediction concentrates on hard timesteps (high noise) where bone reconstruction matters most.



A7. Cross-Attention — K/V from Encoder, Q from Denoiser

Q, K, V projections:

$$Q = W_Q \cdot \text{LayerNorm}(z_{\text{denoiser}}) \in \mathbb{R}^{N \times d_h \cdot h}$$

$$K = W_K \cdot \text{LayerNorm}(f_{\text{enc}}) \in \mathbb{R}^{M \times d_h \cdot h}$$

$$V = W_V \cdot \text{LayerNorm}(f_{\text{enc}}) \in \mathbb{R}^{M \times d_h \cdot h}$$

N = denoiser tokens, M = encoder tokens, h = heads, d_h = head dim.

Asymmetric channels handled: $W_Q \in \mathbb{R}^{C_q \times d}$, $W_K, W_V \in \mathbb{R}^{C_{kv} \times d}$. Allows $C_q \neq C_{kv}$ (e.g. $Q=256\text{ch}$, $K/V=448\text{ch}$ at Down L2).

Attention scores per head:

$$\text{Attn}(Q_h, K_h, V_h) = \text{softmax}\left(\frac{Q_h K_h^\top}{\sqrt{d_h}}\right) V_h$$

Why $\sqrt{d_h}$? Without scaling, $QK^\top$ has variance $\propto d_h$ for unit-variance inputs $\rightarrow$ softmax saturates $\rightarrow$ vanishing gradients.

Cost analysis: $\mathcal{O}(N \cdot M \cdot d)$. At Down L1 ($\frac{1}{2}$ res): $N = M = \frac{DHW}{8}$. At Down L2 ($\frac{1}{4}$ res): $N = M = \frac{DHW}{64}$



A8. Window Attention — Complexity Proof

Full attention complexity on a 3D volume of $L = D \cdot H \cdot W$ tokens:

$$\mathcal{O}_{\text{full}}(L) = L^2 \cdot d$$

Window attention with window ws^3 tokens, $n_W = L/ws^3$ windows:

$$\mathcal{O}_{\text{win}}(L) = n_W \cdot (ws^3)^2 \cdot d = L \cdot ws^3 \cdot d$$

Speedup ratio:

$$\frac{\mathcal{O}_{\text{full}}}{\mathcal{O}_{\text{win}}} = \frac{L}{ws^3}$$

Concrete numbers for $32 \times 128 \times 128$ patch, $ws = 4$:

- $L = 524,288$. Full attention: $\sim 2.7 \times 10^{11}$ ops — infeasible.
- Window: $L \cdot 64 = 3.4 \times 10^7$ ops — 8000 $\times$ cheaper.

Trade-off: W-MSA can't see across window boundaries. SW-MSA (cyclic shift) at odd layers fixes this — after 2 layers, every token has touched a window-shifted neighbour. After 4-6 layers, full receptive field via overlap of shifted windows.



A9. Mamba SSM — Continuous & Discrete Form

Continuous-time SSM:

$$h'(t) = Ah(t) + Bu(t)$$

$$y(t) = Ch(t) + Du(t)$$

where $h(t) \in \mathbb{R}^N$, $u(t) \in \mathbb{R}$, $A \in \mathbb{R}^{N \times N}$.

Zero-order hold discretisation (step Δ):

$$\bar{A} = \exp(\Delta A)$$

$$\bar{B} = (\Delta A)^{-1}(\exp(\Delta A) - I) \cdot \Delta B$$

Discrete recurrence:

$$h_t = \bar{A}h_{t-1} + \bar{B}u_t, \quad y_t = Ch_t$$

Mamba's selectivity: make B, C, Δ functions of input u_t :

$$\Delta_t = \tau_\Delta(W_\Delta u_t), \quad B_t = W_B u_t, \quad C_t = W_C u_t$$

— token-dependent dynamics. Hardware-aware parallel scan computes $h_1, \dots, h_L$ in $\mathcal{O}(L \log L)$ with associative scan.

HiPPO initialisation of A : approximates projection onto Legendre polynomials $\rightarrow$ stable long-range memory.

Bidirectionality (TriAxial): $y = (\text{SSM}_{\text{fwd}}(u) + \text{flip}(\text{SSM}_{\text{bwd}}(\text{flip}(u))))/2$.



A10. Loss Functions — Full Formulas

Weighted HU-aware MAE:

$$\mathcal{L}_{\text{wMAE}} = \frac{\sum_i w_i |p_i - t_i|}{\sum_i w_i + \varepsilon}, \quad w_i = \begin{cases} 3.0 & t_i > \text{norm}(300) \\ 1.5 & \text{norm}(-700) < t_i \leq \text{norm}(300) \\ 0.5 & \text{else} \end{cases}$$

where $\text{norm}(\text{HU}) = \frac{\text{HU} + 1024}{1500 + 1024} \cdot 2 - 1$.

3D SSIM Loss:

$$\mathcal{L}_{\text{SSIM}} = 1 - \frac{(2\mu_p\mu_t + C_1)(2\sigma_{pt} + C_2)}{(\mu_p^2 + \mu_t^2 + C_1)(\sigma_p^2 + \sigma_t^2 + C_2)}$$

7^3 Gaussian kernel ($\sigma = 1.5$), $C_1 = 0.01^2$, $C_2 = 0.03^2$.

Gradient (edge) Loss:

$$\mathcal{L}_{\text{Grad}} = \frac{1}{3} \sum_{a \in \{D, H, W\}} \|\nabla_a p - \nabla_a t\|_1$$

Focal Frequency Loss (Jiang et al.):

$$\mathcal{L}_{\text{FFL}} = \frac{1}{N} \sum_{\mathbf{k}} w(\mathbf{k}) \cdot |\hat{p}(\mathbf{k}) - \hat{t}(\mathbf{k})|, \quad w(\mathbf{k}) = \frac{|\hat{p}(\mathbf{k}) - \hat{t}(\mathbf{k})|^\alpha}{\text{mean}(\cdot) + \varepsilon}$$

3D rFFT of pred/target, focal weight $\alpha = 1.0$ targets high-frequency error.



A11. Evaluation Metrics — Definitions

MAE / RMSE:

$$\text{MAE} = \frac{1}{N} \sum_i |p_i - t_i|, \quad \text{RMSE} = \sqrt{\frac{1}{N} \sum_i (p_i - t_i)^2}$$

PSNR (data range = 2 for $[-1, 1]$ normalised volumes):

$$\text{PSNR} = 20 \log_{10} \left(\frac{\text{MAX}}{\sqrt{\text{MSE}}} \right) = 20 \log_{10}(2) - 10 \log_{10}(\text{MSE})$$

Tissue-class HU MAE: compute MAE only over voxels in tissue class:

- Air: $t_i < -700$ HU
- Soft tissue: $-700 \leq t_i \leq 300$ HU
- Bone: $t_i > 300$ HU

Gamma index (clinical dosimetry): for each voxel r_i :

$$\gamma(r_i) = \min_{r_j} \sqrt{\frac{|d_p(r_j) - d_t(r_i)|^2}{\Delta D^2} + \frac{\|r_j - r_i\|^2}{\Delta r^2}}$$

Pass if $\gamma \leq 1$. “2%/2mm” = $\Delta D = 2\%$ dose, $\Delta r = 2$ mm distance. Pass rate = % voxels passing.

Why dose metrics? SynthRAD challenge report: image metrics (MAE, SSIM) don't fully predict clinical dose accuracy.



A12. Training Setup — AdamW & Mixed Precision

AdamW vs Adam (decoupled weight decay):

Adam couples weight decay with gradient updates:

$$\theta_{t+1} = \theta_t - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \varepsilon}} - \eta \lambda \theta_t$$

AdamW decouples them:

$$\theta_{t+1} = \theta_t - \eta \cdot \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t + \varepsilon}} + \lambda \theta_t \right)$$

— weight decay is independent of the per-parameter scaling of Adam → better generalisation.

CosineAnnealingWarmRestarts: $T_0 = 100$, $T_{\text{mult}} = 2$. LR schedule:

$$\eta_t = \eta_{\min} + \frac{\eta_{\max} - \eta_{\min}}{2} \left(1 + \cos\left(\frac{T_{\text{cur}}}{T_i} \pi\right) \right)$$

Restarts at $t = 100, 300, 700, \dots$ (T_i doubles each cycle).

Why $T_0 = 100$? Loss stage change at epoch 50 must NOT coincide with LR restart → avoids gradient shock and NaN.

AMP fp16: mixed precision via `torch.cuda.amp`. fp32 master weights, fp16 activations & gradients.

GradScaler prevents fp16 underflow. bfloat16 not used — `torch.fft` (used in FFL) doesn't support bf16.



A13. Expert Q&A — Diffusion Choices

Q: Why DDPM not DDIM? DDPM is stochastic; we run 3 MC samples and average to recover variance. DDIM is deterministic — single pass, no MC needed — but loses the calibrated uncertainty IDDPM provides via v . Future work: use DDIM for $3\times$ inference speedup once we drop MC averaging.

Q: Why batch size 1? 3D patches at $32 \times 128 \times 128$ + Mamba/Swin activations + AMP fp16 = $\sim 14\text{--}18$ GB peak on 24 GB GPU. Batch=2 OOMs. Compensate with gradient accumulation if effective batch matters.

Q: Why concat MRI at input, not cross-attn for baseline? Concat is the simplest possible conditioning and serves as the lower bound. Baseline establishes that even minimal conditioning + powerful denoiser works. Hybrid model demonstrates the gain from richer cross-attn injection.

Q: How does the model see MRI at every step? MRI is concat'd with x_t at every step. At $t = 900$ (near pure noise), x_t is essentially noise — MRI is the only meaningful signal guiding the denoiser. Without MRI conditioning, model would generate any plausible CT, not patient-specific.

Q: What does “decode” mean at inference? Network does NOT decode in VAE sense. It *denoises*. Starting from pure noise x_T , iteratively transform to clean x_0 over 50 steps. The U-Net's decoder arm (upsampling) is part of SwinViTModel itself.



A14. Expert Q&A — Mamba & Encoder

Q: Why frozen MRI encoder? Pretraining objective (dual-task L1 + SSIM) is aligned with anatomical fidelity, not denoising. Fine-tuning during diffusion training would distort the anatomical features. Freezing preserves them.

Q: Why only f_2, f_3, f_4 for cross-attn? Why skip f_1 ? Cost is $\mathcal{O}(N \cdot M)$. At full res $N \approx D \cdot H \cdot W$ — prohibitive. Also f_1 (64ch) carries low-level edges; the U-Net's own ResBlocks recover these locally. Cross-attn at f_1 adds compute without information.

Q: Why concat $f_3 \parallel f_4$ at Down L2 instead of two separate cross-attns? Single cross-attn with concatenated K/V is cheaper than two sequential cross-attns. f_4 (256ch $\frac{1}{8}$ res) provides global anatomy summary; f_3 (192ch $\frac{1}{4}$ res) provides local structure. Both needed at deepest decoder. Trilinear upsample f_4 to f_3 's resolution before concat.

Q: Why bidirectional Mamba? Standard Mamba is causal — token at position i only sees positions $1, \dots, i$. For images, future context is just as informative as past. Forward + flipped backward scan, average $\rightarrow$ bidirectional context. Cost: $2\times$ vs unidirectional.

Q: How does TriAxial differ from VMamba's cross-scan? VMamba scans 2D images in 4 directions ($\rightarrow, \leftarrow, \downarrow, \uparrow$). TriAxial scans 3D volumes along 3 *independent* axes (D, H, W) with separate SSMs per axis. SegMamba scans the full 3D volume in one flat sequence.



A15. Expert Q&A — Loss & Training

Q: Why predict $\hat{x}_0$ if you also output v ? Aren't they coupled? No. $\mathcal{L}_{\text{simple}}$ supervises $\hat{x}_0$ with stop-gradient on v . $\mathcal{L}_{\text{VLB}}$ supervises v with stop-gradient on $\hat{x}_0$. They're trained jointly but independently.

Q: Why is $\lambda_{\text{VLB}} = 0.001$ so small? VLB is a loose bound; without scaling it dominates and training becomes unstable. Empirically 0.001 is the largest value that doesn't disrupt L_1 training.

Q: Why bone weight $\times 3$? Bone is $\sim 4\%$ of voxels but contributes $\sim 50\%$ of total error. Uniform L1 under-fits bone $\rightarrow$ smoothed skull edges $\rightarrow$ poor dose accuracy. $\times 3$ weighting empirically rebalances.

Q: Why ramp-in losses linearly over 30 epochs? Hard-switching from wMAE-only to wMAE+SSIM+Grad+DS at fixed epoch caused gradient shock $\rightarrow$ NaN at epoch 52. Linear ramp $w_t = \min(1, (t - t_0)/30)$ smoothly introduces each component.

Q: Why FFL ramp starts at epoch 200, not 50? FFL targets high-frequency content. Early in training, structure is unstable — FFL would amplify noise. Wait until structure converges (epoch 200) before sharpening high frequencies.

Q: Why CosineAnnealingWarmRestarts $T_0 = 100$, not 50? $T_0 = 50$ would coincide with the loss stage change (also at epoch 50) $\rightarrow$ LR jumps + loss change simultaneously $\rightarrow$ NaN. $T_0 = 100$ deliberately separates these events.



A16. Expert Q&A — Clinical & Evaluation

Q: PSNR vs SSIM vs Bone MAE vs Gamma — which matters most? For clinical use: *Gamma pass rate at 2%/2mm* is the gold standard — measures dose accuracy in radiation therapy. PSNR/SSIM are convenient but the SynthRAD report shows they don't always correlate with dose accuracy. Bone MAE matters specifically because bone attenuation drives most of the dose calculation.

Q: Why is TriMamba V2 best at PSNR/SSIM but Diffusion UMamba best at Gamma? Image metrics reward overall pixel similarity. Dose metrics reward correct attenuation in clinically-relevant regions. Diffusion's stochastic sampling + tissue-weighted losses produce HU values that integrate correctly through Beer-Lambert, even when individual voxels differ from ground truth. TriMamba minimises voxel error; Diffusion UMamba minimises dose error.

Q: How does sliding-window inference work at boundary? $32 \times 128 \times 128$ patches with 50% overlap. Output is averaged in the overlap regions (count tensor tracks number of patches per voxel). Avoids hard patch-boundary artifacts.

Q: What about distribution shift across centres? SynthRAD2023 has multi-centre data (different scanners, sequences). Currently: train pooled, test pooled. Future: test-time encoder adaptation via masked MRI reconstruction → adapter layers fine-tune at deployment without CT.

Q: Computational cost per inference? Hybrid Diffusion: 50 steps $\times$ U-Net forward + 1 frozen encoder forward, ~ 15 min for full $192 \times 192 \times 96$ volume on RTX 3090. TriMamba V2 (deterministic): single pass ~ 45 sec.



A17. Architecture Specs — Side-by-Side

	Baseline Swin	Hybrid (Pretrain Enc.)	TriMamba V2	Diffusion UMamba
Backbone	SwinViTModel	SwinViTModel + MRIEnc	TriMambaUNet	DDPM with Mamba
Diffusion	DDPM (IDDPM)	DDPM (IDDPM)	— (deterministic)	DDPM (IDDPM)
Conditioning	MRI concat input	Cross-attn + FiLM	MRI concat input	MRI concat input
Patch size	$64 \times 64 \times 4$	$64 \times 64 \times 4$	$32 \times 128 \times 128$	$128 \times 128 \times 16$
Channels (base)	64	64	32	64
Levels	4	4 + Encoder 4-level	4	4
Total params	~45M	55.14M (39.10M trainable)	19.38M	52.84M
Norm	GroupNorm32	GroupNorm32	InstanceNorm3d	GroupNorm32
Activation	SiLU	SiLU	LeakyReLU(0.2)	SiLU
Optimiser	AdamW	AdamW	AdamW	AdamW
LR	1×10^{-4}	1×10^{-4}	2×10^{-4}	1×10^{-4}
Schedule	Cosine	Cosine	CosineWR ($T_0 = 100$)	Cosine
Batch size	1	1	1	1
Loss	L1 + VLB	L1 + VLB	CompoundLossV2	L1 + VLB
Epochs	500	500	500	500
Inference steps	50 (respaced)	50 (respaced)	N/A (single pass)	50 (respaced)
MC samples	3	3	1	3



A18. Variational Lower Bound (VLB) — Full Derivation

VLB on $-\log p_\theta(x_0)$:

$$\mathcal{L}_{\text{VLB}} = \underbrace{D_{\text{KL}}(q(x_T | x_0) \| p(x_T))}_{L_T} + \sum_{t>1} \underbrace{D_{\text{KL}}(q(x_{t-1} | x_t, x_0) \| p_\theta(x_{t-1} | x_t))}_{L_{t-1}} - \underbrace{\log p_\theta(x_0 | x_1)}_{L_0}$$

L_T : fixed (no learnable parameters), constant in training.

L_{t-1} : KL between two Gaussians, closed form:

$$L_{t-1} = \frac{1}{2} \left[\frac{\|\tilde{\mu}_t - \mu_\theta\|^2}{\sigma_\theta^2} + \log \frac{\sigma_\theta^2}{\tilde{\beta}_t} + \frac{\tilde{\beta}_t}{\sigma_\theta^2} - 1 \right]$$

L_0 : discrete log-likelihood for the final reconstruction step.

Practical: hybrid loss

$$\mathcal{L}_{\text{hybrid}} = \mathcal{L}_{\text{simple}} + \lambda \cdot \mathcal{L}_{\text{VLB}}, \quad \lambda = 0.001$$

— $\mathcal{L}_{\text{simple}}$ trains $\hat{x}_0$ (mean), VLB term trains v (variance). Stop-gradients prevent each term from interfering with the other.



A19. Python Libraries — Stack Used

Deep Learning Core:

- torch 2.8.0 — model definition, AMP, autograd
- torchvision 0.23.0 — transforms, basic vision ops
- torchmetrics 1.8.2 — PSNR / SSIM during training
- mamba-ssm + causal-conv1d — Mamba SSM kernels
- timm — attention/transformer building blocks

Medical Imaging:

- monai 1.5.1 — 3D medical-imaging primitives, sliding-window inference
- nibabel 5.3.2 — NIfTI / medical volume I/O
- simpleitk 2.5.2 — registration, resampling
- pydicom — DICOM read for SynthRAD raw data

Numerical / Scientific:

- numpy 2.2.6 — arrays, FFT, linalg
- scipy 1.16.2 — gamma analysis, dose convolutions
- scikit-image 0.25.2 — SSIM, PSNR (skimage.metrics)
- scikit-learn 1.7.2 — splits, metrics
- sympy 1.14.0 — symbolic schedule derivation

Visualisation:

- matplotlib 3.10.6 — training dashboards, slice plots
- plotly 6.5.0 — interactive volume views
- napari 0.6.6 — 3D volume browsing
- pillow 11.3.0 — image I/O
- tiffiff — TIFF stacks

Data & Utilities:

- pandas 2.3.2 — metric tables, dosimetric CSVs
- tqdm 4.67.1 — training progress bars
- PyYAML 6.0.3 — config files
- joblib 1.5.2 — parallel preprocessing
- opencv-python 4.12.0 — aux 2D ops

Specialised Loss / Eval:

- lpips 0.1.4 — perceptual loss (ablation)
- piq 0.8.0 — IQA metrics (FSIM, MS-SSIM)
- ptflops 0.7.5 — FLOP counting

Environment: CUDA 11.8, Python 3.10.20



A20. Appendix Summary — Quick Reference

- **A0:** Exact parameter counts for all six model families and hybrid-training stages.
- **A2–A3:** Forward $q(x_t | x_0)$ closed form. Reverse posterior $\tilde{\mu}_t, \tilde{\beta}_t$.
- **A4:** learn_sigma — v interpolates $\log \beta_t \leftrightarrow \log \tilde{\beta}_t$. $\lambda_{\text{VLB}} = 0.001$.
- **A5:** Timestep resampling $1000 \rightarrow 50$, rescale_timesteps to keep sinusoidal embed input range.
- **A6:** x_0 -pred vs ε -pred algebraically equivalent but loss weighting differs $\rightarrow x_0$ -pred better for medical.
- **A7:** Cross-attn Q/K/V with separate projections handle asymmetric channels ($C_q \neq C_{kv}$).
- **A8:** Window attention $\mathcal{O}(L \cdot ws^3)$ vs full $\mathcal{O}(L^2)$ — 8000 $\times$ cheaper at $ws = 4$.
- **A9:** Mamba SSM continuous-time + ZOH discretisation + selective B, C, Δ + HiPPO init.
- **A10:** wMAE, SSIM, Grad, FFL formulas with weights.
- **A11:** PSNR for $[-1, 1]$ range, tissue-class HU MAE, Gamma index ≤ 1 pass criterion.
- **A12:** AdamW (decoupled weight decay), CosineWR ($T_0 = 100$ to avoid loss-stage collision).
- **A13–A16:** Q&A on diffusion / Mamba / encoder / loss / clinical eval.
- **A17:** Architecture spec comparison table.
- **A18:** VLB derivation $L_T + \sum L_{t-1} - L_0$.

